

Travelling Salesman Problem

Three Graph Problems

- Eulerian Tour (ET): Polynomial time solvable
- Hamiltonian Cycle (HC): NP-Complete
- Travelling Salesman Problem (TSP): NP-Hard

Eulerian Tour

- An Eulerian Tour is path in the graph that uses each edge in the graph exactly once.
- Such a tour exists if and only if the degree of each vertex is even.
 - In this case a tour can be found in polynomial time, $O(|V|+|E|)$.

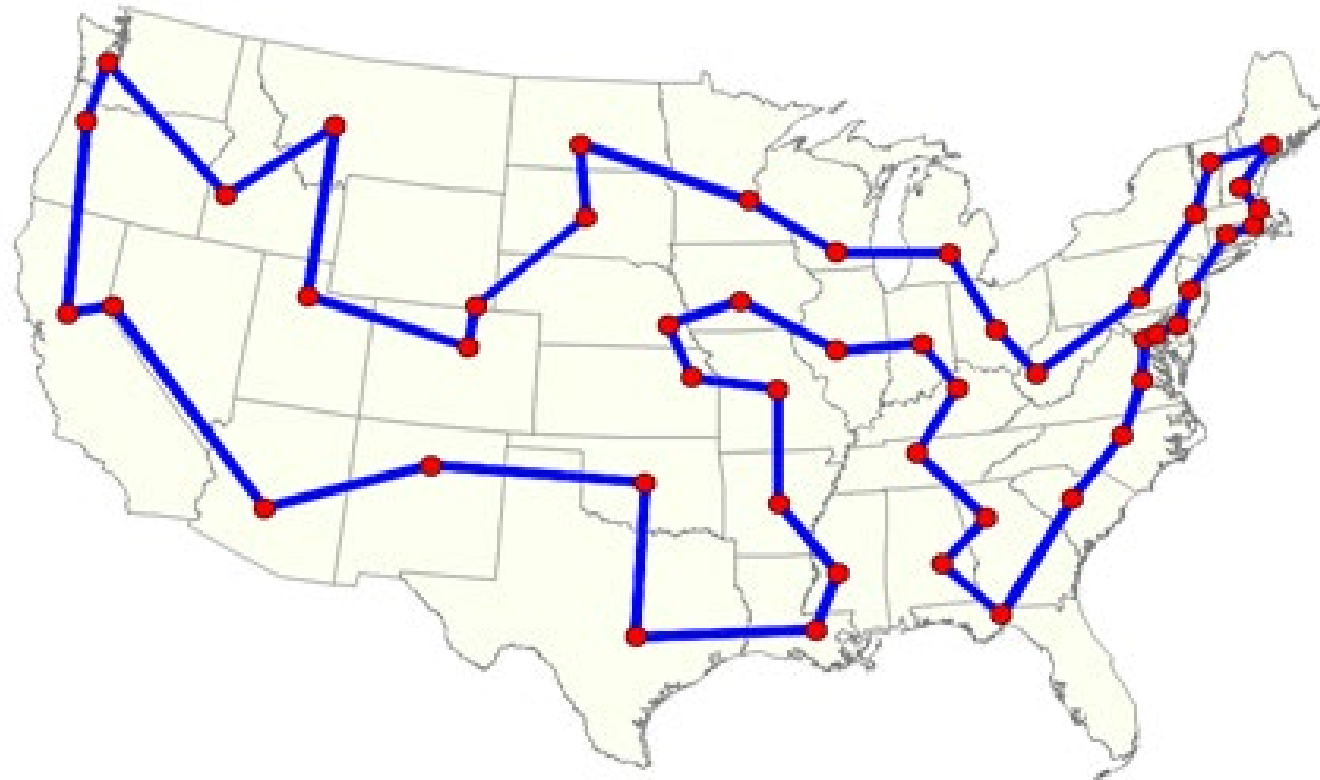
Hamiltonian Cycle

- A Hamiltonian Cycle is a tour (cycle) in G passing through every vertex exactly once.
- The problem asks, does a graph contain a Hamiltonian Cycle?
- NP-Complete

Travelling Salesman Problem

- let $G = (V, E)$ be a complete graph such that every edge $e \in E$ has a cost, $c(e)$.
- The cost of a set of edges $E' \subseteq E$ is $\text{cost}(E') = \sum_{e \in E'} c(e)$. The cost of a Hamiltonian cycle T is then $c(T)$, the sum of the costs of its edges.
- The problem asks to find a Hamiltonian cycle, T , with minimal cost among all cycles.
- NP-Hard (reduction from HC) .
- ... and hard to approximate.

Travelling Salesman Problem



Hardness of Approximation

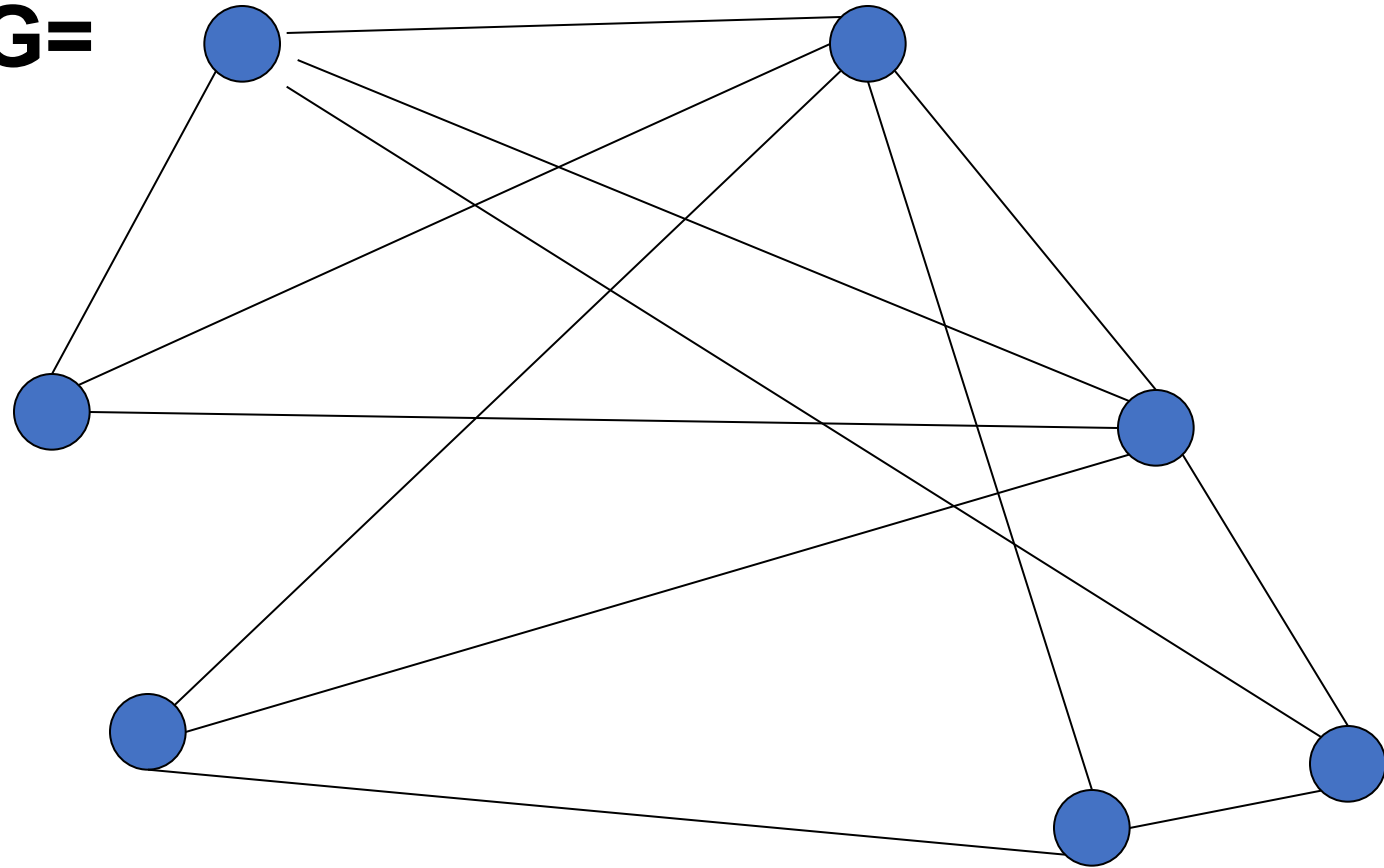
Claim: For every $c > 1$, there is no polynomial time algorithm which can approximate TSP within a factor of c , unless $P = NP$.

Proof

- By reduction from Hamiltonian cycle.
 - Given a graph G , we want to determine if it has a HC.
 - Construct a complete graph G' with same vertices as G , where each edge e has weight 1 if it is in G , and weight $c*n$ if it is not (n is the number of vertices).

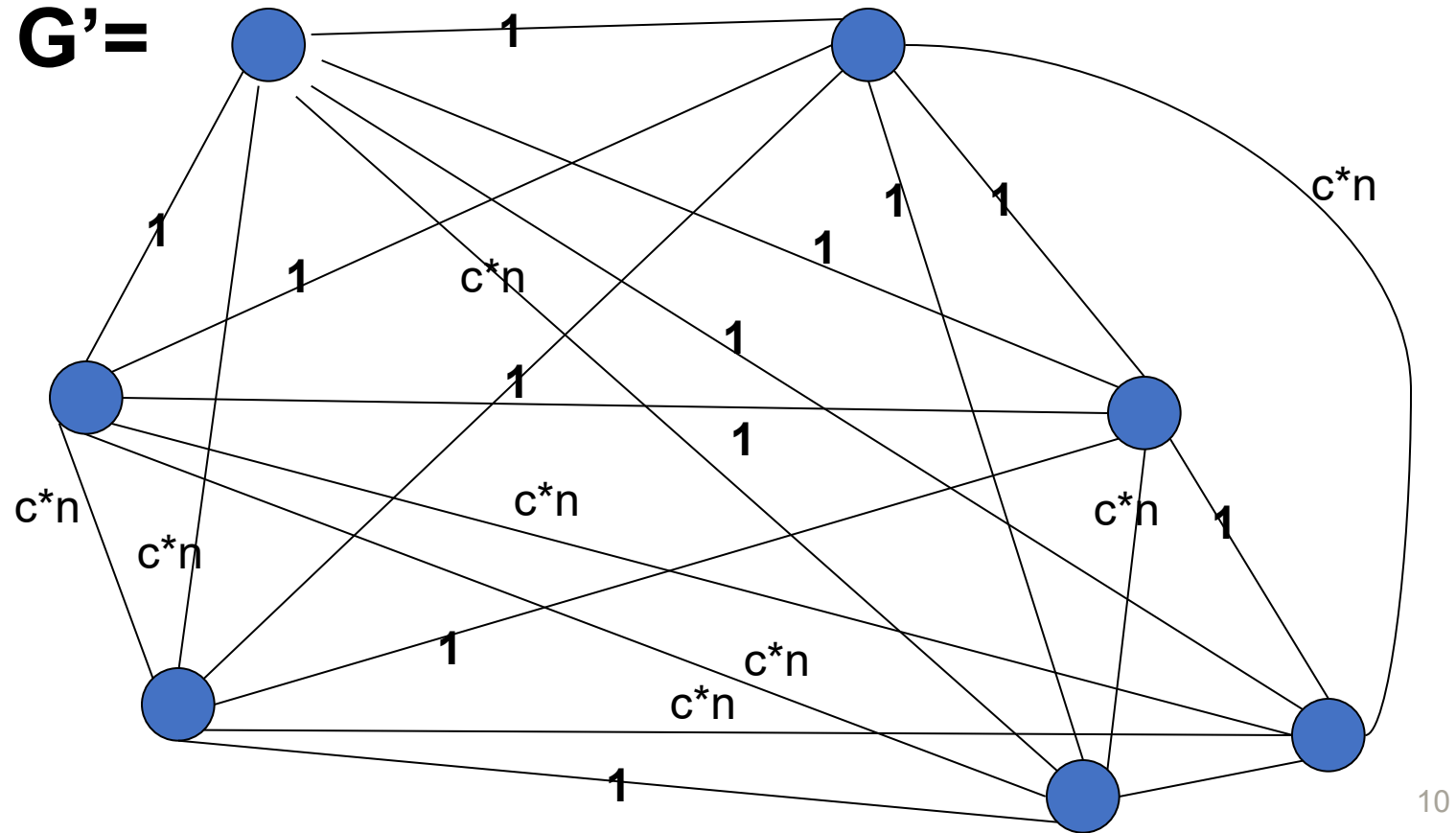
Example

G=



Example

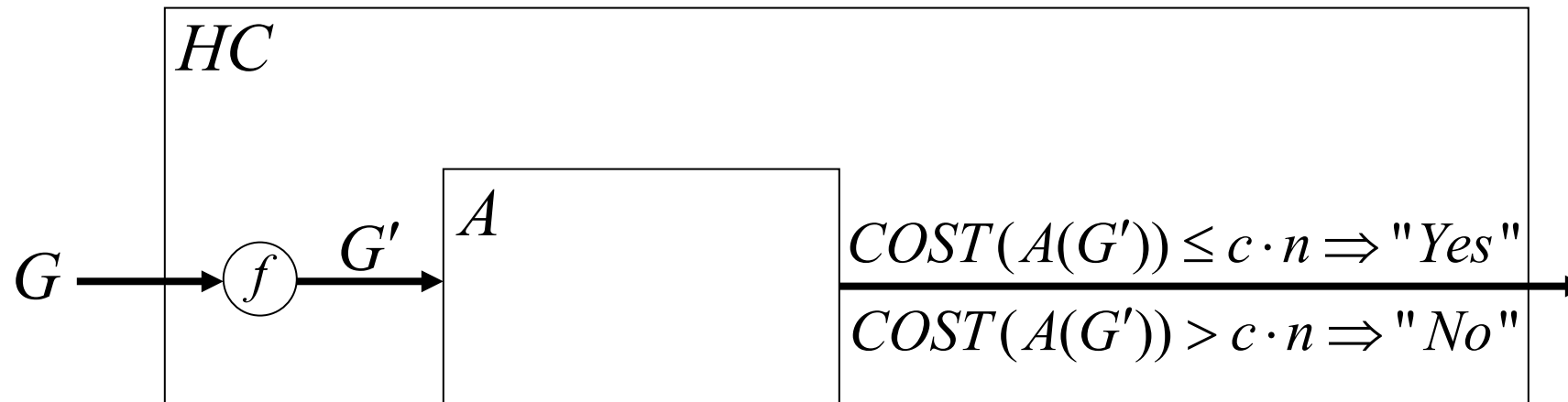
G' =



Proof

- Run algorithm A for solving TSP within a factor of c .

-



- If there is a HC, TSP has a solution of weight n , and approximation is at most $c \cdot n$
- If there is no HC, then every tour in TSP has at least one edge of weight $c \cdot n$, so weight of tour is at least $c \cdot n + n - 1$.

Metric

- A (complete) graph with weight function w on the edges is called a metric if:
 - For any two vertices u, v in the graph:
$$w(u, v) = w(v, u)$$
 - The Triangle Equality holds in the graph.

Triangle Inequality

- The triangle inequality holds in a (complete) graph with weight function w on the edges if for any three vertices u, v, x in the graph:

$$w(u, v) \leq w(u, x) + w(x, v)$$

- Metric TSP is still NP-hard, but now we can approximate.

Approximation Algorithms for TSP

- Greedy Algorithms for the TSP
 - Nearest-neighbor algorithm
 - Multifragment-heuristic algorithm
- Minimum-Spanning-Tree-Based Algorithms
 - Twice-around-the-tree algorithm
 - Christofides Algorithm
- Local Search Heuristics

Greedy Algorithms for the TSP:

Nearest-neighbor algorithm

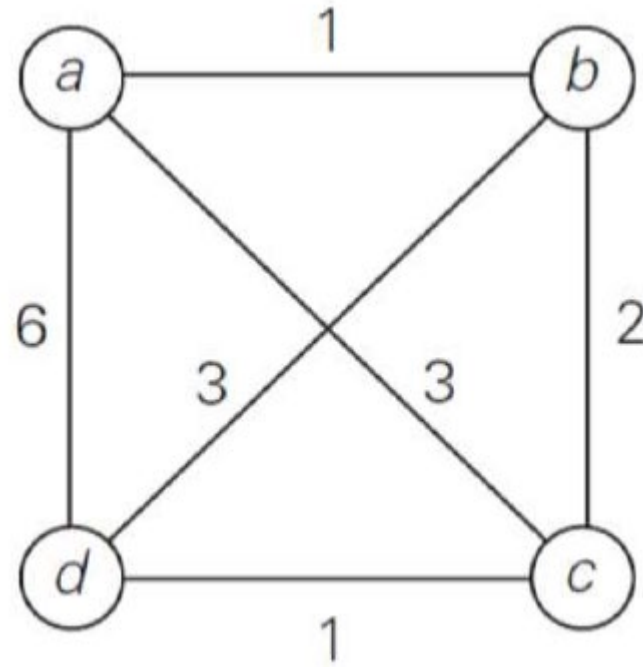
- Nearest-neighbor algorithm
 - Step 1 Choose an arbitrary city as the start.
 - Step 2 Repeat the following operation until all the cities have been visited: go to the unvisited city nearest the one visited last (ties can be broken arbitrarily).
 - Step 3 Return to the starting city.

Nearest-neighbor algorithm

With a as the starting vertex,
the nearest-neighbor algorithm
yields the tour (Hamiltonian
cycle) sa :

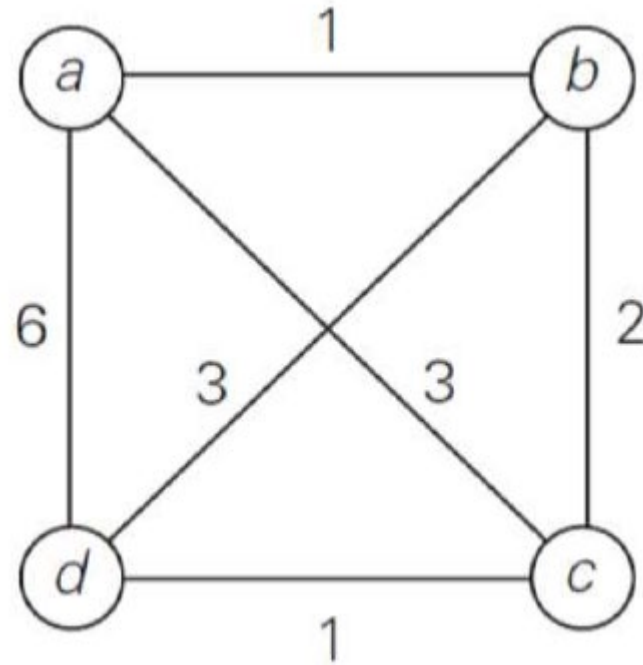
$a - b - c - d - a$ of length 10.

An optimal solution can be s^* :
 $a - c - d - b - a$ of length 8.



Nearest-neighbor algorithm

- The ratio in this example is
- $\frac{10}{8} = 1.25$
- For the metric TSP, the approximation is
 $\frac{1}{2} (\lceil \log_2 n \rceil + 1)$



Greedy Algorithms for the TSP:

Multifragment-heuristic algorithm

- Consider the problem of finding a minimum-weight collection of edges in a given complete weighted graph so that all the vertices have degree 2.

Multifragment-heuristic algorithm

- **Step 1**

- Sort the edges in increasing order of their weights. (Ties can be broken arbitrarily.)
- Initialize the set of tour edges to be constructed to the empty set.

- **Step 2**

- Repeat this step n times, where n is the number of cities in the instance being solved:
- Add the next edge on the sorted edge list to the set of tour edges, provided this addition **does not create a vertex of degree 3 or a cycle of length less than n** ;
- Otherwise, skip the edge.

- **Step 3**

- Return the set of tour edges.

Multifragment-heuristic algorithm

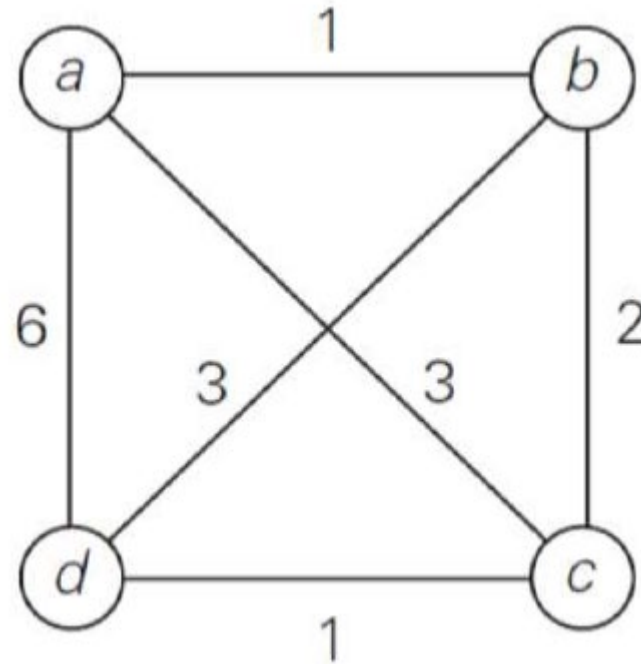
Sorted Edges as per weights:

$\{(a,b), (c,d), (b,c), (a,c), (b,d), (a,d)\}$

$\{1, 1, 2, 3, 3, 6\}$

- Initial Tour Set = $\{\}$
- Inset edge to Tour Set = $\{(a,b)\}$ - OK
- Inset edge to Tour Set = $\{(c,d)\}$ - OK
- Inset edge to Tour Set = $\{(b,c)\}$ - OK
- Inset edge to Tour Set = $\{(a,c)\}$ - **NO**
- Inset edge to Tour Set = $\{(b,d)\}$ - **NO**
- Inset edge to Tour Set = $\{(a,d)\}$ - OK

$a - b - c - d - a$ of length 10.



Multifragment-heuristic algorithm

- For the metric TSP, the approximation is

$$\frac{1}{2} (\lceil \log_2 n \rceil + 1)$$

Minimum-Spanning-Tree-Based Algorithms

- There are approximation algorithms for the traveling salesman problem that exploit a connection between Hamiltonian circuits and spanning trees of the same graph.
- Removing an edge from a Hamiltonian circuit yields a spanning tree, so we use Min Span Tree to produce a TSP tour

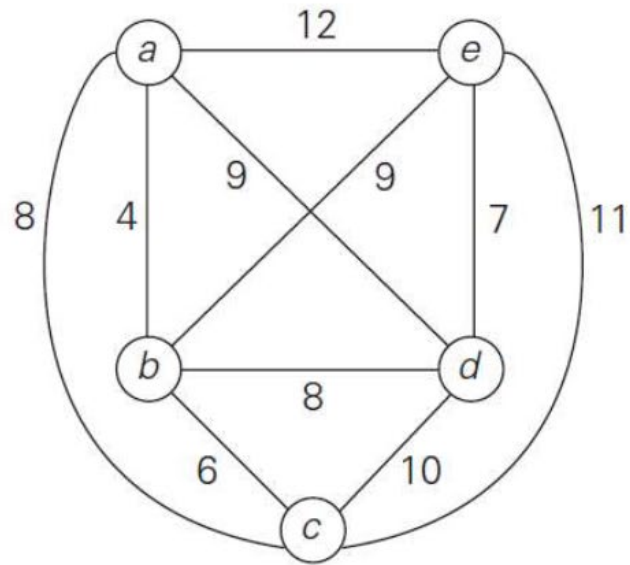
Minimum-Spanning-Tree–Based Algorithms

Twice-around-the-tree algorithm

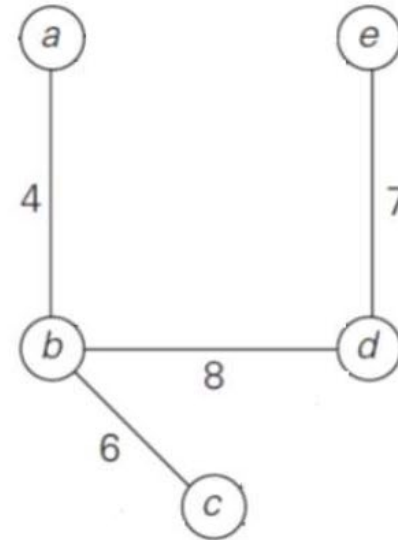
- Given G , construct an MST T for G
- For each edge in T create a double edge. (This will guarantee that the degrees of all the vertices are even)
- Find an Euler tour in the doubled T .
- Create shortcuts in the Euler tour, to create a tour.
 - Traverse the Euler tour.
 - Whenever the Euler tour returns to a vertex already visited, “skip” that vertex.
 - The process creates a Hamiltonian cycle.

Twice-around-the-tree algorithm

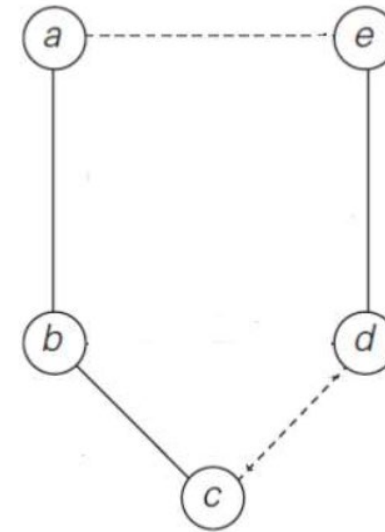
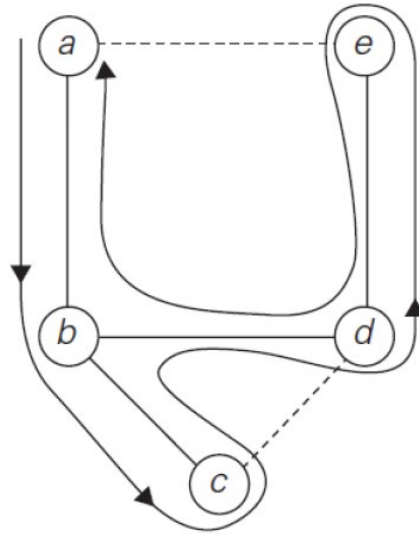
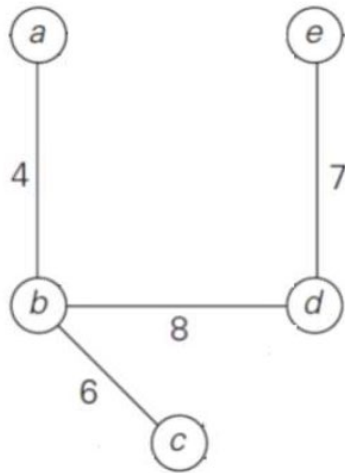
Initial Graph



Minimum Spanning Tree



Twice-around-the-tree algorithm



TSP : Cost = $4+6+10+7+12 = 39$

2-approximation

Claim: The above algorithm gives a 2-approximation for the TSP problem (in a metric graph with the triangle inequality).

Proof:

Definitions:

- OPT is the optimal solution (set of edges) for TSP.
- A is the set of edges chosen by the algorithm.
- EC is the set of edges in the Euler cycle.
- T is the set of edges in the MST

2-approximation

Proof Continued:

- $\text{cost}(T) \leq \text{cost}(\text{OPT})$:
 - since OPT is a cycle, remove any edge and obtain a spanning tree.
- $\text{cost}(\text{EC}) = 2\text{cost}(T) \leq 2\text{cost}(\text{OPT})$.
- $\text{cost}(A) \leq \text{cost}(\text{EC})$
 - A is created by taking “shortcuts” from EC. (These are indeed shortcuts, because of the triangle inequality.)
- So $\text{cost}(A) \leq 2\text{cost}(\text{OPT})$. $\square$

Minimum-Spanning-Tree–Based Algorithms

Christofides Algorithm

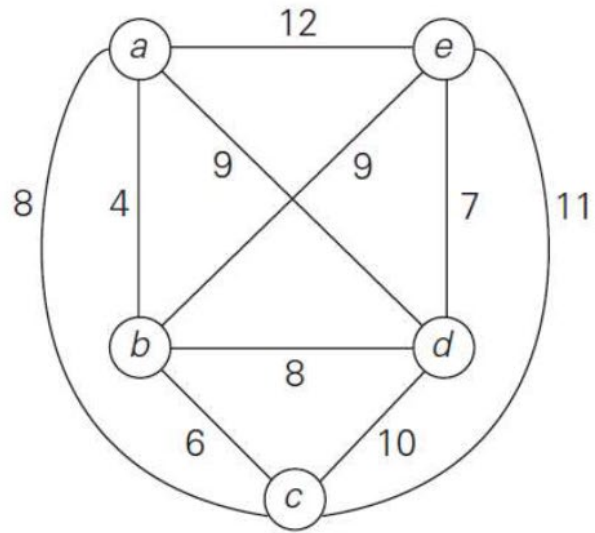
- Professor Nicos Christofides extended the 2.0 TSP and published that the worst-case ratio of the extended algorithm was $3/2$.

Christofides Algorithm

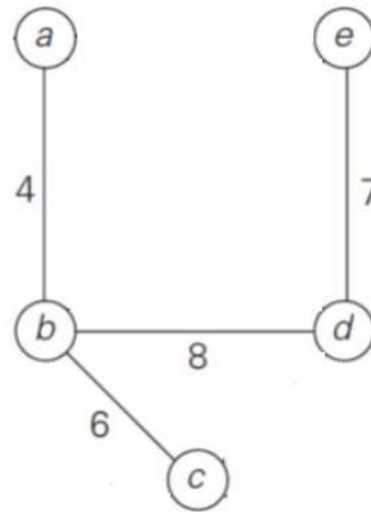
- Algorithm:
 1. Compute MST graph T .
 2. Compute a minimum-weighted matching graph M between all odd-degree vertices in T .
 3. Combine T and M as edge set and compute an Euler tour.
 4. Traverse each vertex taking shortcuts to avoid visited nodes.

Christofides Algorithm

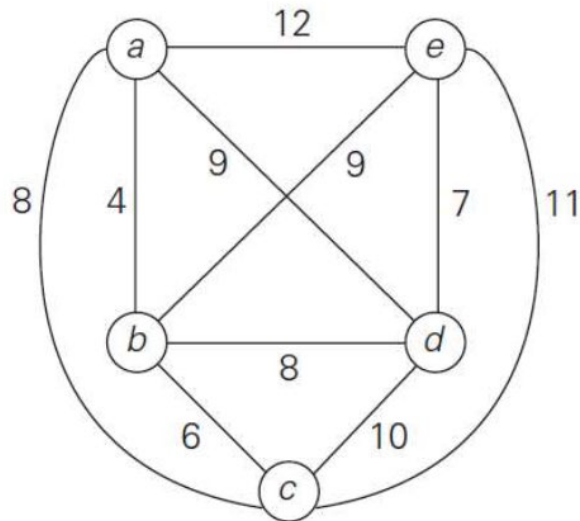
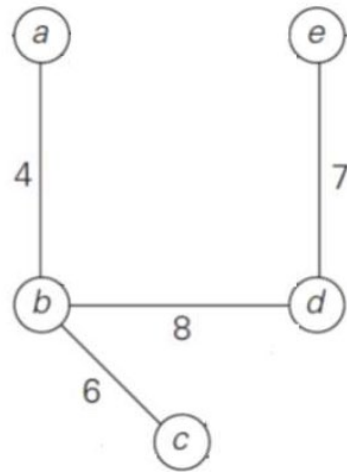
Initial Graph



Minimum Spanning Tree



Christofides Algorithm



- Odd degree edges:

a, b, c, e

- Possible matching pairs with cost are :

$$(a,b) \text{ \& } (c,e) = 4 + 11 = 15$$

$$(a,c) \text{ \& } (b,e) = 8 + 9 = 17$$

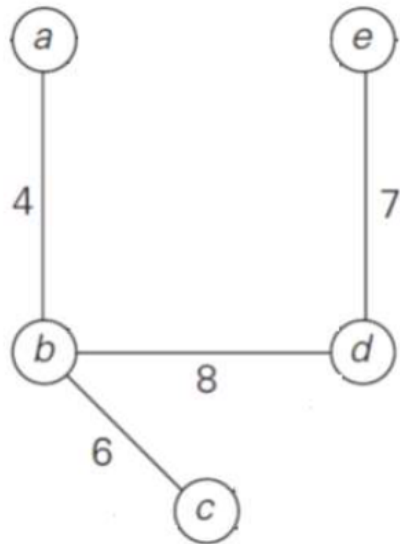
$$(a,e) \text{ \& } (b,c) = 12 + 6 = 18$$

- Minimum of it is:

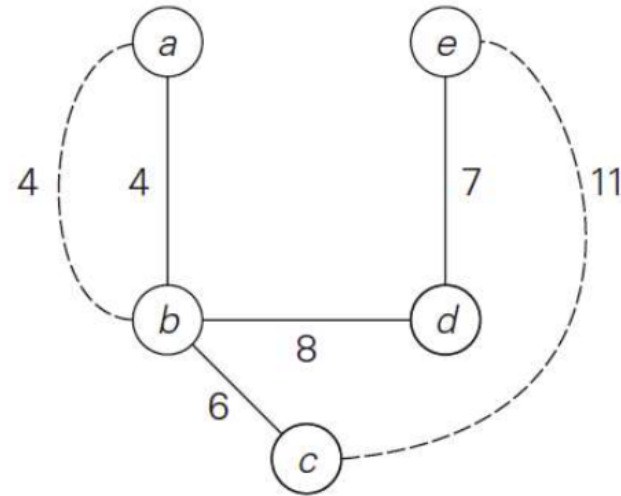
$$(a,b) \text{ \& } (c,e) = 4 + 11 = 15$$

Christofides Algorithm

Minimum Spanning Tree



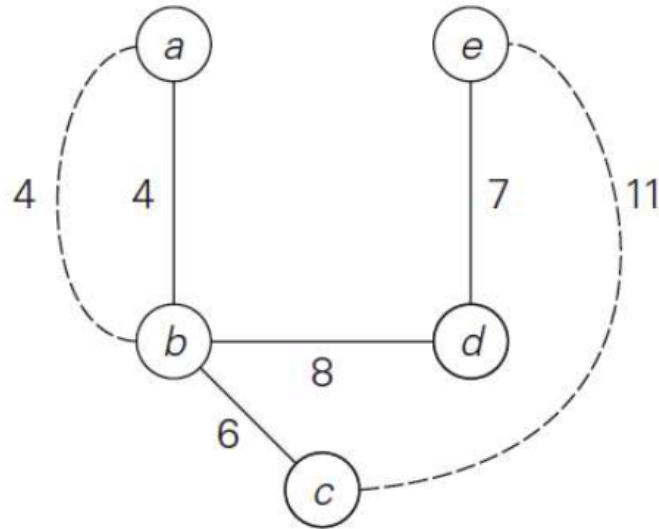
Matching Pair Added to MST



Christofides Algorithm

Matching Pair Added to MST

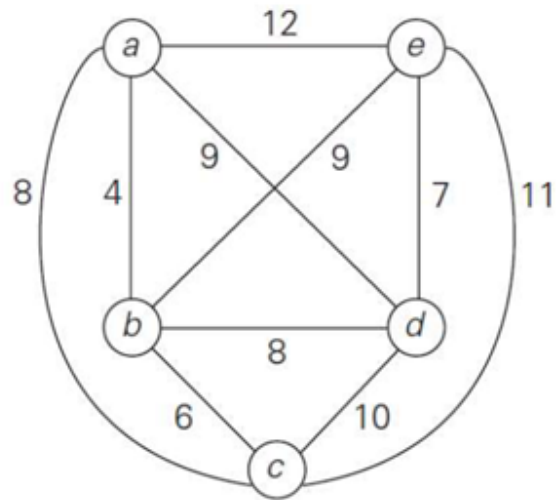
Euler Tour / Euler Circuit



$a - b - c - e - d - b - a$

Christofides Algorithm

Initial Graph

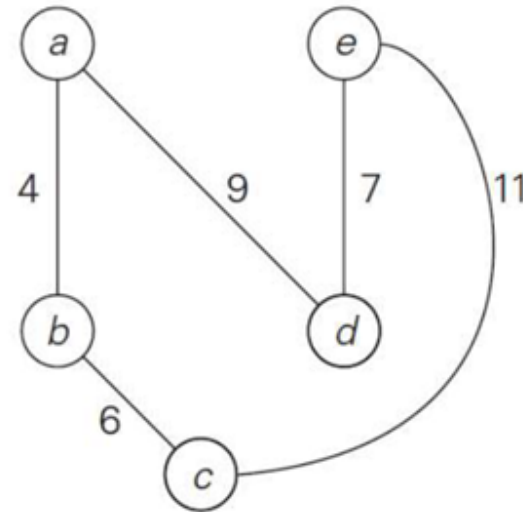


Cut Short - Eliminate Repetition

a - b - c - e - d - **b** - a

a - b - c - e - d - a

Hamiltonian Circuit



TSP : Cost = 4+6+11+7+9 = 37

1.5-approximation

Claim: The above algorithm gives a 1.5-approximation for the TSP problem (in a metric graph with the triangle inequality).

Proof: Denote:

- OPT, A, and EC as before.
- M is the set of edges in the **minimum** matching.
- T is the set of edges in the MST.

1.5-approximation

Proof Continued:

- $\text{cost}(M) \leq \text{cost}(\text{OPT})/2$
 - The minimum tour on any subset B of vertices has a cost of at most $\text{cost}(\text{OPT})$
 - A Hamiltonian cycle in B can be extracted from OPT by “skipping” vertices not in B , resulting in a tour lighter than OPT , due to the triangle inequality.
 - If B is of even size then divide the minimum tour on B into two disjoint matchings of B .
 - The lighter matching of the two (denote by M') has weight at most $\text{cost}(\text{OPT})/2$.

1.5-approximation

Proof Continued:

- $\text{cost}(M) \leq \text{cost}(M') \leq \text{cost}(\text{OPT})/2$.
 - The number of odd vertices in a graph is even. (Always, but Why?)
 - Choose B to be the set of odd vertices in T .
 - M' is not necessarily the minimum matching of B , so $\text{cost}(M) \leq \text{cost}(M')$.

1.5-approximation

Proof Continued:

- EC is an Euler Cycle in $M \cup T$.
- $\text{cost}(\text{EC}) = \text{cost}(T) + \text{cost}(M) \leq 1.5\text{cost}(\text{OPT})$.
- $\text{cost}(A) \leq \text{cost}(\text{EC}) \leq 1.5\text{cost}(\text{OPT})$

